How to Download, Build, and Install MB-System

David W. Caress
October 13, 2026

MB-System is an open source software package written for Unix-like operating systems, and is distributed for free as source code for download. In order to install MB-System, prerequisites need to be installed and the source distribution must be downloaded, unpacked, compiled, linked, and installed locally. The necessary steps are specific to different operating systems, so are presented separately below, after more general advice. Lastly, see the "Additional Setup" section for steps that may be required for modules to be found by MB-System. If you are updating an existing installation, first see the section "Updating an MB-System Installation".

These instructions are also available on the MB-System web site:
https://www.mbari.org/technology/mb-system/installation/

How to Download the MB-System Source Distribution

The source code for MB-System is available from a repository on Github:
https://github.com/dwcaress/MB-System

This link will open the page for the most recent stable release of MB-System. The source code distribution can be downloaded as zip or tar.gz archives from the "Assets" section at the bottom.
https://github.com/dwcaress/MB-System/releases/latest

This link will open a page listing all recent MB-System releases, with the most recent release at the top. The most recent release could be a beta or pre-release. The source code distribution can be downloaded as zip or tar.gz archives from the "Assets" section at the bottom.
https://github.com/dwcaress/MB-System/releases

This link will download the current state of the master branch of the MB-System repository:
https://github.com/dwcaress/MB-System/archive/refs/heads/master.zip

General Advice for Building and Installing MB-System

MB-System now includes two methods for building and installing the software, which are presented as options in the instructions. The recommended method is using the CMake package (which is not available

for all operating systems). The other method is based on GNU Autotools. The CMake build system is new to MB-System with version 5.7.9, and is intended to replace the older Autotools approach for current and future operating systems. The old Autotools build system will continue to be included in MB-System distributions for the foreseeable future to enable building and installation on legacy operating systems. The Autotools build system will not be maintained to enable its use on current or future operating systems, and the CMake build system will not be modified to support building on legacy systems.

The sequence of commands by which one builds and installs a package using CMake is very different from Autotools. For CMake, one creates a new directory to hold the package build (commonly named "build" by convention) in the top level of the source structure, then cd's into that directory, and then executes **cmake** with the argument "..", which provides **cmake** the path to the top level of the source directory. CMake then copies the source into a build tree and generates Makefiles in that tree to enable compilation, linkage, and installation, again using the program **make**.

The GNU Autotools build system includes a script named **configure** at the top of the distribution structure which, when executed, generates a set of files named "Makefile" throughout the structure that hold compilation and linkage instructions used by the program **make**. These compilation and linkage instructions vary depending on the operating system and which versions of prerequisite software packages are installed, and in some cases special arguments must be included in the configure command. Once configure has been successfully executed, running the program **make** at the top level then actually invokes the local compiler and linker to build and install MB-System. Several Autotools programs, including autoconf, automake, and libtool, are used to generate the configure script included in each distribution. This build system had been the sole means by which MB-System could be built since 2011.

MB-System depends on a number of other software packages. For some operating systems, special arguments to the configure script are required to integrate the MB-System source to prerequisite software packages.

Among the software packages that are prerequisite for MB-System are:

- GMT (Generic Mapping Tools)
- Proj
- GDAL
- netCDF
- FFTW (Fastest Fourier Transform in the West)
- X11
- OpenMotif
- OpenGL
- OpenCV (for optional photomosaicing tools)
- PCL (Point Cloud Library) (planned, not yet in release 5.8.0)

The sections below provide instructions for building MB-System on a few common operating systems, including MacOs, Ubuntu Linux, Debian Linux, CentOs Linux, and the CygWin environment on Windows. These instructions include the installation of the prerequisite software packages using a package manager relevant to each OS and the special arguments needed for the cmake command or the configure script.

Note that a bonafide 3-button mouse is necessary for operations within the GUIs for viewing grids and editing and navigation adjustment of survey data.

MacOS (26 Tahoe, 15 Sequoia; earlier versions)

MB-System can now be built and installed on MacOs Sequoia or Tahoe using either the Homebrew or MacPorts package managers to install the prerequisite software packages; Homebrew is recommended. For earlier MacOS versions, MacPorts is necessary. System setup is the same, either way.

System Setup (Xcode and X11)

Complete these steps after upgrading your MacOS and before installing MB-System. See also "Updating an MB-System Installation" below for guidance on how to clean things up before upgrading to the next version.

- From the Apple App Store, install Xcode developer tools, which includes the LLVM compiler suite. In order for the compiler and linker to be operated from a terminal command line, also install the Xcode Command Line Tools using the following command in a terminal:

  ```
  xcode-select --install
  ```

- Install the XQuartz X11 server from
  https://www.xquartz.org
  XQuartz 2.8.5 or later is required for all MB-System installations.

Possible compiler issue with Xcode: If you have an Xcode Command Line Tools installation and update to version 16 (probably following updating your Mac to MacOs 15 Sequoia), you may find that the clang++ compiler now fails to find the standard C++ header files. This breaks any ability to compile C++ code in MB-System, and also breaks updating of many MacPorts packages. According to the MacPorts Problem Hotlist (https://trac.macports.org/wiki/ProblemHotlist#clts16) this is due to the Command Line Tools installer neglecting to remove a directory that was installed by older versions. You can fix the problem by running the following command in any shell:

```
sudo rm -rf /Library/Developer/CommandLineTools/usr/include/c++
```

Compiling should then proceed normally.

Note that if MB-System is otherwise working properly, errors related to X11 or Motif can be ignored (such as "Locale not supported for XmbText...").

Option 1: Install prerequisites and MB-System using Homebrew

- Follow the instructions for installing Homebrew on the home page for the project at https://brew.sh/. The package can be installed from the command line or by using an available .pkg file.
- To access the "tap" that pours mbsystem:

```
brew update
brew tap dwcaress/mbsystem
brew trust dwcaress/mbsystem
```

- To install the current stable MB-System formula (package), and all prerequisite formulae (packages):

```
brew install mbsystem
```

To install the most recent beta MB-System formula, and all prerequisite formulae:

```
brew install mbsystem-beta
```

Option 2: Install prerequisites using Homebrew and then manually install MB-System

- Follow the instructions for installing Homebrew on the home page for the project at https://brew.sh/. The package can be installed from the command line or by using an available .pkg file.
- Use Homebrew to install the following formulae (packages) required to build and then run MB-System:

```
brew install cmake pkg-config fftw gdal netcdf gmt \
  ghostscript proj libx11 libxt openmotif \
  mesa mesa-glu opencv qt@6 vtk
```

Note: as of September 15, 2026, the Homebrew package mesa (version 26.2.2) is broken such that the MB-System programs mbgrdviz, mbeditviz, and mbnavadjust do not display correctly. We have established formulae of the prior working version (mesa 26.1.4) and the closely linked mesa-glu (version 9.0.3) within the private tap dwcaress/mbsystem. Until this problem is eliminated by a new version of mesa that works properly, MB-System can be built and run successfully by accessing the private tap and installing the older mesa and mesa-glu (always use mb-mesa and mb-mesa-glu together):

```
brew tap dwcaress/mbsystem
brew trust dwcaress/mbsystem
brew install cmake pkg-config fftw gdal netcdf gmt \
  ghostscript proj libx11 libxt openmotif \
  dwcaress/mbsystem/mb-mesa \
  dwcaress/mbsystem/mb-mesa-glu \
  opencv qt@6 vtk
```

- Download and unpack the MB-System source as described in "Download and Install MB-System" below.

- Build option 1: **CMake Build System** (recommended)
  From the top of the MB-System source tree, create a working directory named "build", cd into "build", and execute cmake. When using the mb-mesa and mb-mesa-glu formulae, add the option -DmacosUseMbMesa=ON:

```
mkdir build
cd build
cmake .. -DmacosUseMbMesa=ON
```

- Build option 2: **Autotools Build System** (not recommended)
  With Homebrew, configure finds all of the prerequisites without location options: PROJ, FFTW, NetCDF, GDAL, GMT and OpenCV through pkg-config and the packages' config scripts, X11 and Motif in the Homebrew installation (/opt/homebrew), and OpenGL in the mb-mesa and mb-mesa-glu formulae (used only when both are installed, never mixed with the standard Homebrew mesa or mesa-glu). Any location given explicitly with a --with-... option takes precedence.

```
./configure --prefix=/usr/local --enable-test
```

  To also build the OpenCV based photomosaicing tools and the Terrain Relative Navigation tools:

```
./configure --prefix=/usr/local --enable-test \
  --enable-mbtrn --enable-mbtnav --enable-opencv
```

- Finally, make and install MB-System:

```
make
sudo make install
```

- Set the location of the MB-System library in your home directory's gmt.conf file - edit directly or execute:

```
pushd ~ ; gmt set GMT_CUSTOM_LIBS "/opt/homebrew/lib/mbsystem.dylib" ; popd
```

(This is the location used by the Homebrew mbsystem formula; for a manual build installed in /usr/local, see "Additional Setup" below.)

Option 2: Install prerequisites using MacPorts and then manually install MB-System

- Install MacPorts using the appropriate downloadable installer package from: https://www.macports.org/install.php
- After MacPorts installation, first make sure the default port packages are current by running selfupdate and then install the MB-System prerequisite packages.

```
  sudo port -v selfupdate
  sudo port install gmt6 gdal-hdf5 fftw-3 mesa libGLU openmotif xorg opencv4
pkgconfig
```

MacPorts occasionally publishes a broken `mesa` release (for example, 26.2.2). If MB-System's graphical utilities fail to build or run after a `mesa` update, you can force MacPorts to stay on (or revert to) a working version:

```
  # If an older mesa is still installed but inactive, just reactivate it:
  port installed mesa
  sudo port activate mesa @<old-version>_<revision>

  # Otherwise, pull an older Portfile and serve it from a local port repository
  # that takes precedence over the default MacPorts source, then reinstall:
  git clone --depth 1 https://github.com/macports/macports-ports.git /tmp/
macports-ports-old
  cd /tmp/macports-ports-old
  git log --oneline -- graphics/mesa/Portfile   # find a commit before the
broken bump
  git checkout <good-commit> -- graphics/mesa/Portfile
  sudo mkdir -p /opt/local/var/macports/local_ports/graphics/mesa
  sudo cp graphics/mesa/Portfile /opt/local/var/macports/local_ports/graphics/
mesa/
  cd /opt/local/var/macports/local_ports && sudo portindex
  # Add "file:///opt/local/var/macports/local_ports" above the rsync line in
  # /opt/local/etc/macports/sources.conf, then:
  sudo port -f uninstall mesa
  sudo port install mesa
```

Either way, pin the version so a later `sudo port upgrade outdated` does not pull the broken release back in:

```
  sudo port pin mesa
```

Run `sudo port unpin mesa` once a fixed `mesa` release is available upstream.

- Also make sure that a current version of Python3 is available. First list the available Python3 versions, install the most recent, and then set port to link that version to python3:

```
port select --list python
```

Versions of Python available through MacPorts (April 14, 2025):

  - python27
  - python311
  - python312
  - python313

The most recent version is python313, so install it:

```
sudo port install python313
sudo port select --set python python313
sudo port select --set python3 python313
```

Also install the most recent Python imaging library Pillow:

```
sudo port install py313-Pillow
```

Download and Install MB-System

Once the prerequisite software is fully installed, you can proceed with the MB-System installation.

- Download the MB-System source package from the repository at GitHub:
  https://github.com/dwcaress/MB-System
  There are often beta releases that are more recent than the current stable release. The most recent beta release will be at the top of this page:
  https://github.com/dwcaress/MB-System/releases
  We recommend using Chrome for the download rather than Safari, because Chrome does not prematurely unpack the tarball.

- Copy the MB-System distribution tarball to /usr/local/src (or somewhere else, if you prefer - this is our convention for consistency when managing many installations) and unpack it there. If you do not have write privilege in /usr/local/src/ then you will need to execute the following commands using sudo. Replace the filename with the name of the current release.

```
mv MB-System-5.8.1.tar.gz /usr/local/src
cd /usr/local/src
tar xvzf MB-System-5.8.1.tar.gz
```

and then cd into the top directory of the resulting structure:

```
cd MB-System-5.8.1
```

If you installed the prerequisites with Homebrew (Option 2 above), build and install using the commands given there. If you used MacPorts, continue below.

- Build option 1: **CMake Build System** (recommended)
  At that location (/usr/local/src/MB-System-5.8.1/), create a working directory named "build", **cd** into "build", and then execute **cmake** at the location /usr/local/src/MB-System-5.8.1/build/. The cmake command should successfully enable building the entire current MB-System (5.7.9 or later) on any Mac computer with the prerequisites installed using MacPorts (this has been tested with computers running Monterey through Ventura). Finally, make and install MB-System.

```
mkdir build
cd build
cmake ..

sudo make
sudo make install
```

- Build option 2: **Autotools Build System** (not recommended)
  At that location (/usr/local/src/MB-System-5.8.1/), execute the configure script, named **configure**, with the options necessary for your context. The Xcode compiler tools do not look for header files or libraries in the locations used by MacPorts, and so it is necessary to specify these locations for several of the prerequisite packages. This command should successfully enable building the current core MB-System (5.7.9 or later) on any Mac computer with the prerequisites installed through MacPorts. This has been tested with computers running Ventura and Monterey.

```
./configure \
  --prefix=/usr/local \
  --disable-static \
  --enable-shared \
  --enable-hardening \
  --enable-test \
  --with-proj-lib=/opt/local/lib/proj9/lib \
  --with-proj-include=/opt/local/lib/proj9/include \
  --with-gmt-config=/opt/local/lib/gmt6/bin \
  --with-fftw-lib=/opt/local/lib \
  --with-fftw-include=/opt/local/include \
  --with-x11-lib=/opt/local/lib \
  --with-x11-include=/opt/local/include \
  --with-motif-lib=/opt/local/lib \
  --with-motif-include=/opt/local/include \
  --with-opengl-include=/opt/local/include \
  --with-opengl-lib=/opt/local/lib \
  --with-otps-dir=/usr/local/src/otps
```

The MB-System codebase includes some components that are optional when using the Autotools build system, such as OpenCV based photomosaicing (enabled with --enable-opencv) and a realtime Terrain Relative Navigation infrastructure and toolset (--enable-mbtrn and --enable-mbtnav). This configure command should enable building the entire MB-System package, including these optional tools:

```
./configure \
  --prefix=/usr/local \
  --disable-static \
  --enable-shared \
  --enable-hardening \
  --enable-test \
  --with-proj-lib=/opt/local/lib/proj9/lib \
  --with-proj-include=/opt/local/lib/proj9/include \
  --with-gmt-config=/opt/local/lib/gmt6/bin \
  --with-fftw-lib=/opt/local/lib \
  --with-fftw-include=/opt/local/include \
  --with-x11-lib=/opt/local/lib \
  --with-x11-include=/opt/local/include \
  --with-motif-lib=/opt/local/lib \
  --with-motif-include=/opt/local/include \
  --with-opengl-include=/opt/local/include \
  --with-opengl-lib=/opt/local/lib \
  --enable-mbtrn \
  --enable-mbtnav \
  --enable-opencv \
  --with-opencv-include=/opt/local/include/opencv4 \
  --with-opencv-lib=/opt/local/lib/opencv4 \
  --with-otps-dir=/usr/local/src/otps
```

Finally, make and install MB-System:

```
sudo make
sudo make install
```

Ubuntu Noble Numbat 24.04, Jammy Jellyfish 22.04 and Focal Fossa 20.04

This example is relevant for the Ubuntu 20.04LTS, 22.04LTS, and 24.04LTS distributions because the prerequisite package names are the same and the entirety of the current MB-System can be built.

Ubuntu and Debian Linux distributions include multiple tools for managing software distributions. We recommend using the command line program apt to install the prerequisite software for MB-System.

- Install Ubuntu 20, 22 or 24 from an ISO image of the release (https://ubuntu.com/download/desktop).
- Update the starting packages:

```
sudo apt upgrade
```

- Install compilers and other essential software:

```
sudo apt install build-essential
```

- Install MB-System prerequisites:

```
sudo apt install libtirpc-dev \
  netcdf-bin libnetcdf-dev libgdal-dev \
  gmt libgmt6 libgmt-dev libproj-dev \
  libfftw3-bin libfftw3-dev libmotif-dev \
  xfonts-100dpi libglu1-mesa-dev \
  libopencv-dev cmake gfortran
```

MB-System also requires Python3 and the Pillow library for Python3. Both of these packages are installed by default in Ubuntu 20, 22 and 24.

Log out and log back in after installing the prerequisites so that the X11 server will load updated resources such as fonts.

- Download the MB-System source package from the repository at GitHub:
  https://github.com/dwcaress/MB-System
  There are often beta releases that are more recent than the current stable release. The most recent beta release will be at the top of this page:
  https://github.com/dwcaress/MB-System/releases
  Although MB-System can be installed from a source tree located anywhere, we generally place the tarball in /usr/local/src and then unpack and build MB-System there.

- Unpack the MB-System distribution tarball in /usr/local/src/ (or somewhere else, if you prefer - this is our convention for consistency when managing many installations). If you do not have write privilege in /usr/local/src/ then you will need to execute the following commands using sudo. (Replace the filename with the name of the current release.)

```
mv MB-System-5.8.0.tar.gz /usr/local/src
cd /usr/local/src
tar xvzf MB-System-5.8.0.tar.gz
```

and then cd into the top directory of the resulting structure:

```
cd MB-System-5.8.0
```

- Option 1: **CMake Build System** (recommended)
  At that location (/usr/local/src/MB-System-5.8.0/), create a working directory named "build", **cd** into "build", and then execute **cmake** at the location /usr/local/src/MB-System-5.8.0/build/. The cmake

command should successfully enable building the entire current MB-System (5.7.9 or later) on Ubuntu 20, 22 or 24 as long as the prerequisite software packages have been installed. Finally, install MB-System.

```
mkdir build
cd build
cmake ..

make
sudo make install
```

- Option 2: **Autotools Build System** (not recommended)
  At that location (/usr/local/src/MB-System-5.8.0/), execute the configure script, named **configure**, with the following options to build the entire MB-System package.

```
./configure \
  --enable-mbtrn --enable-mbtnav --enable-opencv \
  --with-opencv-include=/usr/include/opencv4 \
  --with-opencv-lib=/lib/x86_64-linux-gnu
```

The MB-System libraries will be placed in /usr/local/lib/, but the runtime dynamic linker does not look for shared libraries in this directory by default. Users can add /usr/local/lib to the directories searched for shared libraries by adding "/usr/local/lib" to the environment variable LD_LIBRARY_PATH. This is accomplished by placing the command

```
export LD_LIBRARY_PATH=/usr/local/lib:$LD_LIBRARY_PATH
```

in an environment file in the user's home directory named .zprofile (if using the zsh shell) or .profile (if using the bash shell). Other user environment files can be used, such as .zshrc for zsh or .bashrc for bash. The preferred alternative to using LD_LIBRARY_PATH is to embed the shared library location in the compiled libraries and executables by passing an rpath command to the linker by setting the LDFLAGS environment variable available to the configure command:

```
LDFLAGS="-Wl,-rpath -Wl,/usr/local/lib" \
  ./configure \
    --enable-mbtrn --enable-mbtnav --enable-opencv \
    --with-opencv-include=/usr/include/opencv4 \
    --with-opencv-lib=/lib/x86_64-linux-gnu
```

(Note that the CMake build system, Option 1, builds the binaries such that the runtime libraries are reliably found without any additional commands or arguments.)

Finally, install MB-System:

```
make
sudo make install
```

Ubuntu Bionic Beaver 18.04

Ubuntu 18 has a version of CMake that is too old to work with MB-System's CMake build system. Therefore MB-System can only be built and installed on Ubuntu 18 using the old Autotools based build system. Neither the photomosaicing nor the Terrain Relative Navigation tools can be built under Ubuntu 18 because the versions of gcc and OpenCV are too old.

Ubuntu and Debian Linux distributions include multiple tools for managing software distributions. We recommend using the command line program apt to install the prerequisite software for MB-System.

- Install Ubuntu 18 from an ISO image of the release (https://ubuntu.com/download/desktop).
- Update the starting packages:

```
sudo apt upgrade
```

- Install compilers and other essential software:

```
sudo apt install build-essential
```

- Install MB-System prerequisites:

```
sudo apt install gmt libgmt5 libgmt-dev gmt-common \
  proj-bin proj-data libproj-dev libgdal-dev \
  libfftw3-3 libfftw3-dev libmotif-dev \
  xfonts-100dpi libglu1-mesa-dev gfortran
```

  MB-System also requires Python3 and the Pillow library for Python3. Both of these packages are installed by default in Ubuntu 18.

  Log out and log back in after installing the prerequisites so that the X11 server will load updated resources such as fonts.

- Download the MB-System source package from the repository at GitHub:
  https://github.com/dwcaress/MB-System
  There are often beta releases that are more recent than the current stable release. The most recent beta release will be at the top of this page:
  https://github.com/dwcaress/MB-System/releases

Although MB-System can be installed from a source tree located anywhere, we generally place the tarball in /usr/local/src and then unpack and build MB-System there.

- Unpack the MB-System distribution tarball in /usr/local/src/ (or somewhere else, if you prefer - this is our convention for consistency when managing many installations). If you do not have write privilege in /usr/local/src/ then you will need to execute the following commands using sudo. (Replace the filename with the name of the current release.)

```
mv MB-System-5.8.0.tar.gz /usr/local/src
cd /usr/local/src
tar xvzf MB-System-5.8.0.tar.gz
```

and then cd into the top directory of the resulting structure:

```
cd MB-System-5.8.0
```

- Only option for this operating system: **Autotools Build System**
At that location (/usr/local/src/MB-System-5.8.0/), execute the configure script, named **configure**, without options to build MB-System:

```
./configure
```

The MB-System libraries will be placed in /usr/local/lib/, but the runtime dynamic linker does not look for shared libraries in this directory by default. Users can add /usr/local/lib to the directories searched for shared libraries by adding "/usr/local/lib" to the environment variable LD_LIBRARY_PATH. This is accomplished by placing the command

```
export LD_LIBRARY_PATH=/usr/local/lib:$LD_LIBRARY_PATH
```

in an environment file in the user's home directory named .zprofile (if using the zsh shell) or .profile (if using the bash shell). Other user environment files can be used, such as .zshrc for zsh or .bashrc for bash. The preferred alternative to using LD_LIBRARY_PATH is to embed the shared library location in the compiled libraries and executables by passing an rpath command to the linker by setting the LDFLAGS environment variable available to the configure command:

```
LDFLAGS="-Wl,-rpath -Wl,/usr/local/lib" \
  ./configure
```

- Once the makefiles have been generated by configure, build and install MB-System using:

```
make
sudo make install
```

Debian

This example is relevant for Debian 11 through 13.

Ubuntu and Debian Linux distributions include multiple tools for managing software distributions. We recommend using the command line program apt to install the prerequisite software for MB-System.

- Install Debian from an ISO image of the release downloaded from https://www.debian.org.
- Update the starting packages:

```
sudo apt upgrade
```

- Install compilers and other essential software:

```
sudo apt install build-essential
```

- Install MB-System prerequisites:

```
sudo apt install netcdf-bin libnetcdf-dev libgdal-dev \
  gmt libgmt6 libgmt-dev libproj-dev \
  libfftw3-bin libfftw3-dev libmotif-dev \
  xfonts-100dpi libglu1-mesa-dev \
  libopencv-dev gfortran
```

  MB-System also requires Python3 and the Pillow library for Python3. Both of these packages are installed by default in Debian 11 and 12.

  Log out and log back in after installing the prerequisites so that the X11 server will load updated resources such as fonts.

- Download the MB-System source package from the repository at GitHub:
  https://github.com/dwcaress/MB-System
  There are often beta releases that are more recent than the current stable release. The most recent beta release will be at the top of this page:
  https://github.com/dwcaress/MB-System/releases
  Although MB-System can be installed from a source tree located anywhere, we generally place the tarball in /usr/local/src and then unpack and build MB-System there.

- Unpack the MB-System distribution tarball in /usr/local/src/ (or somewhere else, if you prefer - this is our convention for consistency when managing many installations). If you do not have write privilege

in /usr/local/src/ then you will need to execute the following commands using sudo. (Replace the filename with the name of the current release.)

```
mv MB-System-5.8.0.tar.gz /usr/local/src
cd /usr/local/src
tar xvzf MB-System-5.8.0.tar.gz
```

and then cd into the top directory of the resulting structure:

```
cd MB-System-5.8.0
```

- Option 1: **CMake Build System** (recommended)
  At that location (/usr/local/src/MB-System-5.8.0/), create a working directory named "build", **cd** into "build", and then execute **cmake** at the location /usr/local/src/MB-System-5.8.0/build/. The cmake command should successfully enable building the entire current MB-System (5.7.9 or later) on Debian 11 through 13 as long as the prerequisite software packages have been installed. Finally, install MB-System.

```
mkdir build
cd build
cmake ..

make
sudo make install
```

- Option 2: **Autotools Build System** (not recommended)
  At that location (/usr/local/src/MB-System-5.8.0/), execute the configure script, named **configure**, with the following options to build the entire MB-System package.

```
./configure \
  --enable-mbtrn --enable-mbtnav --enable-opencv \
  --with-opencv-include=/usr/include/opencv4 \
  --with-opencv-lib=/lib/x86_64-linux-gnu
```

The MB-System libraries will be placed in /usr/local/lib/, but the runtime dynamic linker does not look for shared libraries in this directory by default. Users can add /usr/local/lib to the directories searched for shared libraries by adding "/usr/local/lib" to the environment variable LD_LIBRARY_PATH. This is accomplished by placing the command

```
export LD_LIBRARY_PATH=/usr/local/lib:$LD_LIBRARY_PATH
```

in an environment file in the user's home directory named .zprofile (if using the zsh shell) or .profile (if using the bash shell). Other user environment files can be used, such as .zshrc for zsh or .bashrc for

bash. The preferred alternative to using LD_LIBRARY_PATH is to embed the shared library location in the compiled libraries and executables by passing an rpath command to the linker by setting the LDFLAGS environment variable available to the configure command:

```
LDFLAGS="-Wl,-rpath -Wl,/usr/local/lib" \
  ./configure \
    --enable-mbtrn --enable-mbtnav --enable-opencv \
    --with-opencv-include=/usr/include/opencv4 \
    --with-opencv-lib=/lib/x86_64-linux-gnu
```

(Note that the CMake build system, Option 1 above, builds the binaries such that the runtime libraries are reliably found without any additional commands or arguments.)

Once the makefiles have been generated, install MB-System:

```
make
sudo make install
```

CentOs 7

This example is relevant for CentOs 7, and includes building the core MB-System programs plus the Terrain Relative Navigation tools, but not the photomosaicing tools (which require a more recent version of OpenCV than included in CentOs 7).

CentOs and Red Hat Linux distributions include multiple tools for managing software distributions. We recommend using the command line program yum to install the prerequisite software for MB-System.

- Install CentOs 7 from an ISO image of the release downloaded from https://www.centos.org/download/.
- Update the starting packages:

```
sudo yum upgrade
```

- Install EPEL repo:

```
sudo yum install epel-release
```

- Install MB-System prerequisites:

```
sudo yum install openmotif openmotif-devel \
  fftw fftw-devel netcdf netcdf-devel \
  proj proj-devel gdal-devel gmt gmt-devel gv \
  mesa-libGL mesa-libGL-devel \
  mesa-libGLU mesa-libGLU-devel
```

Log out and log back in after installing the prerequisites so that the X11 server will load updated resources such as fonts.

- Download the MB-System source package from the repository at GitHub:
  https://github.com/dwcaress/MB-System
  There are often beta releases that are more recent than the current stable release. The most recent beta release will be at the top of this page:
  https://github.com/dwcaress/MB-System/releases
  Although MB-System can be installed from a source tree located anywhere, we generally place the tarball in /usr/local/src and then unpack and build MB-System there.

- Unpack the MB-System distribution tarball in /usr/local/src/ (or somewhere else, if you prefer - this is our convention for consistency when managing many installations). If you do not have write privilege in /usr/local/src/ then you will need to execute the following commands using sudo. (Replace the filename with the name of the current release.)

```
mv MB-System-5.8.0.tar.gz /usr/local/src
cd /usr/local/src
tar xvzf MB-System-5.8.0.tar.gz
```

and then cd into the top directory of the resulting structure:

```
cd MB-System-5.8.0
```

- Only option for this operating system: **Autotools Build System**
  At that location (/usr/local/src/MB-System-5.8.0/), execute the configure script, named **configure**, with the following options to build the entire MB-System package.

```
./configure --enable-mbtrn --enable-mbtnav
```

The MB-System libraries will be placed in /usr/local/lib/, but the runtime dynamic linker does not look for shared libraries in this directory by default. Users can add /usr/local/lib to the directories searched for shared libraries by adding "/usr/local/lib" to the environment variable LD_LIBRARY_PATH. This is accomplished by placing the command

```
export LD_LIBRARY_PATH=/usr/local/lib:$LD_LIBRARY_PATH
```

in an environment file in the user's home directory named .zprofile (if using the zsh shell) or .profile (if using the bash shell). Other user environment files can be used, such as .zshrc for zsh or .bashrc for bash. The preferred alternative to using LD_LIBRARY_PATH is to embed the shared library location in the compiled libraries and executables by passing an rpath command to the linker by setting the LDFLAGS environment variable available to the configure command:

```
LDFLAGS="-Wl,-rpath -Wl,/usr/local/lib" \
  ./configure --enable-mbtrn --enable-mbtnav
```

- Once the makefiles have been generated by configure, build and install MB-System using:

```
make
sudo make install
```

CygWin

CygWin is a collection of software tools that augment Windows computers with a Unix-style environment within which one can build, install, and run Unix-y packages like MB-System.

If Cygwin has been installed on a Windows system, then one must install a number of additional prerequisite packages before building MB-System. The package installation is done using the Cygwin setup program **setup-x86_64.exe** rather than through Linux-style package managers like **apt** or **yum**. This example installs the core command line programs of MB-System without any of the graphical tools. For this purpose the prerequisite packages include:

- gcc
- g++
- rpc-devel
- gambas3-devel
- libproj-devel
- libproj12
- libnetcdf-devel
- libnetcdf
- libgdal-devel
- libgdal19
- libfftw3-devel
- libfftw3
- cmake
- make
- fftw

- fftw-devel
- ghostscript
- gv
- libcurl-devel
- libnpcr0
- libnpcr0-devel
- libpcre
- pcre-devel
- openssh
- subversion
- xinit
- zlib
- zlib-devel
- liblapack-devel

Once Cygwin and the prerequisite packages are installed, proceed as follows:

- Download the MB-System source package from the repository at GitHub:
  https://github.com/dwcaress/MB-System
  There are often beta releases that are more recent than the current stable release. The most recent beta release will be at the top of this page:
  https://github.com/dwcaress/MB-System/releases
  Although MB-System can be installed from a source tree located anywhere, we generally place the tarball in /usr/local/src and then unpack and build MB-System there.

- Unpack the MB-System distribution tarball in /usr/local/src/ (or somewhere else, if you prefer - this is our convention for consistency when managing many installations). If you do not have write privilege in /usr/local/src/ then you will need to execute the following commands using sudo. (Replace the filename with the name of the current release.)

  ```
  mv MB-System-5.8.0.tar.gz /usr/local/src
  cd /usr/local/src
  tar xvzf MB-System-5.8.0.tar.gz
  ```

  and then cd into the top directory of the resulting structure:

  ```
  cd MB-System-5.8.0
  ```

- Only option for this operating system: **Autotools Build System**
  At that location (/usr/local/src/MB-System-5.8.0/), execute the configure script, named **configure**, with the following options to build MB-System without any graphical tools:

```
./configure \
  --enable-mbtrn --enable-mbtnav \
  --disable-dependency-tracking --disable-mbtools
```

- Once the makefiles have been generated by configure, install MB-System:

```
make
sudo make install
```

Additional Setup (for all operating systems)

1. PATH environment variable

Unless one specified otherwise, the MB-System executable programs are installed in the directory /usr/local/bin. For MB-System programs to execute from a command line, the /usr/local/bin directory must be included in a user's $PATH environment variable. This is typically accomplished by adding a statement like

```
    export PATH=/usr/local/bin:$PATH
```

in an environment file in the user's home directory. These are often named .zshenv (if using the zsh shell), or .profile (if using the bash shell); other user environment files can be used, such as .zshrc for zsh or .bashrc for bash.

2. Enable MB-System GMT Modules (gmt.conf)

A key prerequisite for MB-System is the software package Generic Mapping Tools, or GMT. The programs mbcontour, mbswath, mbgrdtiff, and mbgrd2obj are actually GMT plug-in modules invoked as commands of the program gmt:

```
    gmt mbcontour (...various arguments...)
    gmt mbswath (...various arguments...)
    gmt mbgrdtiff (...various arguments...)
    gmt mbgrd2obj (...various arguments...)
```

These modules are contained in a dynamically loaded or shared library that is installed into /usr/local/lib. This library is named mbsystem.so on Linux computers and either mbsystem.dylib (when installed using CMake) or mbsystem.so (when installed using Autotools) on MacOS. For GMT to successfully execute these MB-System modules, the location of this library must be known to GMT. GMT looks for the modules using an internal variable GMT_CUSTOM_LIBS loaded from a configuration file named gmt.conf. One usually enables use of the MB-System GMT modules by creating (or altering) a gmt.conf file in the user's home directory using the gmtset module of GMT:

```
gmt gmtset GMT_CUSTOM_LIBS /usr/local/lib/mbsystem.so
```

or

```
gmt gmtset GMT_CUSTOM_LIBS /usr/local/lib/mbsystem.dylib
```

as appropriate. (When MB-System is installed with Homebrew, the library is /opt/homebrew/lib/mbsystem.dylib; see the MacOS section above.)

Some users modify additional GMT parameters using the gmt.conf file, for example, changing the COLOR_NAN parameter to be white so transparency can be set to 255,255,255 easily in a GIS. A description of all of the gmt.conf parameters is at:
https://docs.generic-mapping-tools.org/dev/gmt.conf.html
To see what GMT parameter settings are being used, type:

```
gmt defaults
```

A complexity not mentioned in the gmt.conf manual page is that GMT looks for gmt.conf first in the current working directory, then if not found, looks for it in the users home directory, and if still not found, uses the system defaults. A number of programs and plotting scripts using GMT create a gmt.conf file in the local directory, alter some of the settings in that file, use it, and then delete it before exiting. If one of these programs crashes or is interrupted before the temporary gmt.conf file is deleted, then a stray gmt.conf file is left that will become the default the next time GMT is run in that directory; we recommend deleting these extra gmt.conf files.

3. Perl ForkManager module

Several MB-System programs are written in the scripting language Perl. Three of these (mbm_multiprocess, mbm_multicopy, mbm_multidatalist) utilize a Perl library called ForkManager that enables parallel processing, and thus require that this library be installed. The management of Perl libraries is done with a tool named cpan rather than with the package managers used to install other MB-System prerequisites. Perl and cpan are available on all of the relevant operating systems. In all cases, execute:

```
cpan Parallel::ForkManager
```

4. Tidal correction (OTPS) setup

Tidal corrections can be applied to bathymetry data using the OSU Tidal Prediction Software (OTPS), which is called with the MB-System utility mbotps. This package extracts a local multi-constituent model for an area from a set of global TPXO-atlas single constituent model files, and can be downloaded from https://www.tpxo.net/otps.

What Gets Installed Where

MB-System installations made using these directions will install files into the following locations:

- /usr/local/bin/ Executable programs
- /usr/local/lib/ Shared libraries
- /usr/local/include/ Header files
- /usr/local/share/man/ Unix style manual pages
- /usr/local/share/doc/mbsystem/html/ Web site manual pages
- /usr/local/share/mbsystem/ Resource data files

Updating an MB-System Installation

A: For those upgrading to MacOS Tahoe from previous MacOS versions, remove completely your MacPorts installation and update the Xcode and X11 prerequisites (as above) before installing Homebrew and MB-System.

Search on "uninstall MacPorts" and follow the steps offered by macports.org. Note that when you come to the step to "Remove the rest of MacPorts", it will fail if all of the libraries they list are not installed on your machine. We suggest beginning by executing the first several lines of code, then executing (starting with "sudo rm -rf") each of the lines that name individual libraries.

Remove also old versions of Homebrew by executing code found at the Homebrew GitHub site (use sudo).

B: Update the MB-System release with Homebrew by first updating Homebrew so it knows to look for the current stable and beta release of MB-System, then upgrading things that need to be upgraded, such as MB-System. A critical bug fix that impacts all of the interactive modules is that mb-mesa must be used instead of the mesa that is supplied by Homebrew. Type:

```
brew update
brew upgrade
```

If prompted to increase your "ulimit", type:

```
ulimit -n 512
```

C: Before updating to a new MB-System version installed from GitHub (e.g., with MacPorts or Linux), we recommend uninstalling the previous version first. In the directory for the previous version (e.g., /usr/local/src/MB-System5.7.9/build/ if using CMake, or /usr/local/src/MB-System5.7.9/ if using Autotools), use the following command:

```
    sudo make uninstall
    sudo make clean
```

Then follow the installation instructions appropriate for your operating system, starting with downloading the next version from GitHub.

Docker Container with MB-System

An updated MB-System Docker Image is generated each time that a new release is created in the MB-System Github repository. This Docker is based on Debian 12 Linux, and can be run on MacOs, Linux, and Windows computers. Data present on the host computer's filesystems can be processed using the MB-System programs in the Docker container.

The MB-System docker image is available at
https://hub.docker.com/r/mbari/mbsystem

Documentation is available in the Github repository at:
https://github.com/dwcaress/MB-System/tree/master/docker/README.md
https://github.com/dwcaress/MB-System/tree/master/docker/README.pdf

Packaged MB-System Distributions

There currently are no packaged distributions of MB-System. We are working on establishing them again.